Bhac M1 Documentation

Marie

January 2025

1 Introduction

1.1 Introduction

2 Bhac+M1 structure & methods

2.1 Preprocessor setup:

2.1.1 Preprocessor grid-blocks

The same preprocessor shortcuts are used for M1 as for the rest of Bhac+. There are some most used cases to remember:
ixO^S : the block of output grid cells (without ghost-zones) in all dimensions
ixI^S : the block of input grid cells (with ghost-zones) in all dimensions
ixO^L : the limits (minimal and maximal cell) of a block of output grid cells (without ghost-zones) in all dimensions e.g. ixOmin1, ixOmin2, ixOmax1, ixOmax2
ixI^L : the limits (minimal and maximal cell) of a block of input grid cells (with ghost-zones) in all dimensions e.g. ixImin1, ixImin2, ixImax1, ixImax2
ix^D : a single grid cell in all dimensions (depending on the outer loop in will either include ghost-zones or not) The number of dimensions is ^ND
^C : acceses a specific component (normally number of components is ÑC = 3)

2.1.2 Preprocessor neutrinos

In the makefile one can specify the number of neutrino species via the flag PPFLAGS += -nspecies=3 and also has to include the m1-directory and makefiles related to m1:
#— M1
VPATH += :$(SRC)/m1
-include $(SRC)/m1/makefile
Note that these two things in the makefile are needed even though you don't use M1. This is needed for the correct compilation and linking of objects. It can happen that compilation without M1 fails. To be safe use this in your makefile: # ——-science dir:

-include $(SRC)/makefile_noM1

Whether M1 is used or not is only defined via the # define M1 in the definitions.h file of a test!
Here are some preprocessor shortcuts which are defined in **vacpp.pl**:
^NS : number of species
^KSP : kind of species in a KSP-loop

NOTE:
KSP = 1 : electron neutrino
KSP = 2 : electron antineutrino
KSP = 3 : heavy neutrino

2.2 M1 variables in Bhac+M1:

For each species 5 new variables have to be initialized and evolved. The initialization happens in mod_variables where the variables for the evolution are
nrad^KSP_,
erad^KSP_,
frad^KSP^C_,

$$\tilde{U} := \begin{pmatrix} \sqrt{\gamma}\Gamma\tilde{n} \\ \sqrt{\gamma}E \\ \sqrt{\gamma}F_i \end{pmatrix} \tag{1}$$

and the variables fo the rates:
Q_er^KSP_,
Q_nr^KSP_,
kappa_a^KSP_,
kappa_s^KSP_,
kappa_nr^KSP_,
tau_path^KSP_
have to be initialized. Depending on their type (radiation_var = 7) for evolved vars or (index radiation_impl_var = 8) for rates they are added to the list of index nw, nwflux, nwcons, nprim and also the limits for nm1rad are set accordingly in amrvacpar.
NOTE:
The evolved m1-variables are inside of the w(), wcons(), wprim() array which are part of the ps%pwrad%w() state-array defined in mod_physicaldata. On the other hand for the rates variables a new state-array had to be defined:
type(m1_walloc), target, dimension(ngridshi) :: pwm1,pwrad
type(m1_impl_state), dimension(ngridshi) :: psm1
accessed as psm1%pwrad%wradimpl(). There we also needed new routines for allocation of the states psm1 and pwrad:

- subroutine alloc_m1_pw(pwm1,ixG^L,nwm1min,nwm1max)

- subroutine dealloc_m1_pw(pwm1)
- subroutine alloc_m1_state(psm1,ixG^L)
- subroutine dealloc_m1_state(psm1)

2.3 Methods for M1:

We use **finite_volume** for the evolution of our m1-variables. The reconstruction of the variables happens inside the routine of **upwind**-scheme and the for the fluxes an extended **HLLE**-scheme with corrections in the diffusive limit is used. For the time-evolution an **IMEX-222**-scheme is implemented in **advance**. The IMEX-222 is well described in Section 3.

2.4 M1 flow in Bhac:

3 IMEX-222 Scheme

3.1 IMEX-222 Scheme: Theoretical Overview

IMEX-222 is an **Implicit-Explicit (IMEX)** Runge-Kutta method, meaning it treats different terms in the equations separately:

- **Explicit Terms**: Advection and flux-related terms are solved explicitly.
- **Implicit Terms**: Stiff source terms, such as collisional reactions or diffusion, are solved implicitly.

The scheme consists of two substeps and is characterized by its **Butcher tableau**:

Explicit Part (Advection Terms)

$$\begin{array}{c|cc} 0 & 0 & 0 \\ 1 & 1 & 0 \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

Implicit Part (Stiff Terms)

$$\begin{array}{c|cc} g & g & 0 \\ 1-g & 1-2g & g \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

where $g = 1 - \frac{1}{\sqrt{2}}$.

Using this tableau, the solution at time $n+1$ is computed iteratively as:

$$U^{(0)} = U^{(n)}, \tag{2}$$

$$U^{(i)} = U^{(0)} + \Delta t \sum_{j=1}^{i} a_{ij} X(U^{(j)}) + \Delta t \sum_{j=1}^{\nu} z_{ij} G(U^{(j)}), \tag{3}$$

$$U^{(n+1)} = U^{(0)} + \Delta t \sum_{j=1}^{\nu} w_j X(U^{(j)}) + \Delta t \sum_{j=1}^{\nu} k_j G(U^{(j)}), \tag{4}$$

where:

- $X(U)$ represents the explicit terms,
- $G(U)$ represents the implicit terms.

3.2 Implementation in Bhac+M1

The implementation of IMEX-222 in the provided code follows these steps:

Preallocation of Variables

- Temporary states $ps2$, $ps1$ are allocated to store intermediate results.
- Initial conditions are set by copying the current state pw into $pw2$.

First Implicit Step

Solve the equation $\xi_1 = y^n + \gamma dt F_{im}(\xi_1)$ using:

```
global_implicit_update(gamma_imex222, dt, t, ps2, ps, psm1).
```
(5)

Explicit Advection Step

Compute $ps1 = y^n + dt F_{ex}(\xi_1)$ using:

```
advect1(typefull1, one, idim^LIM, t, t, ps2, ps1, psold, psm1).
```
(6)

Intermediate Updates

- Compute $ps2 = \frac{ps2 - ps}{\gamma}$.
- Compute $ps = y^n + \frac{dt}{2}(F(\xi_1) + F(\xi_2))$.

Second Implicit Step

Solve $\xi_2 = ps1 + \gamma dt F_{im}(\xi_2)$ using:

```
global_implicit_update(gamma_imex222, dt, t, ps2, ps1, psm1).
```
(7)

Final Update

- Compute $ps = y^n + \frac{dt}{2}(F(\xi_1) + F(\xi_2))$.
- Apply boundary conditions using:

$$\texttt{getbc(t+dt, ps1, psCoarse)}. \tag{8}$$

4 Modules

/src/m1 In this section we discuss the modules needed for m1.

4.1 Metric_interface

The modules discussed here:
mod_m1_metric_interface

Since Bhac contains two parallel setups with non-dynamical spacetime (nonDYSP) and dynamical spacetime (DYSP) it was a natural choice to write a **metric module** setting up the metric quantities. The advantage is that we don't have to write two different versions of M1 for dynamical and non-dynamical spacetime. The module **mod_m1_metric_interface** contains the metric-class of type type m1_metric_helper which can be allocated and deallocated in the routines it is needed. Different subroutines can we called as objects of the metric-class. The module contains routines for filling the metric, lowering and raising indices on the block ixOˆS and on a single grid-point ixˆD.

- subroutine init(self, ixˆL)
- subroutine destroy(self)
- subroutine fill_metric(metricH,wprim,x,ixIˆL,ixOˆL)
- subroutine raise(self,ixIˆL,ixOˆL,covec, vec)
- subroutine lower(self,ixIˆL,ixOˆL,vec, covec)
- subroutine raise_ixD(self,ixˆD,covec, vec)
- subroutine lower_ixD(self,ixˆD,vec, covec)

4.2 MOD-M1 and Explicit

The modules discussed here:
mod_m1

The main routines and full explicit part connected to bhac-infrastructure are covered by the module **mod_m1**

- subroutine m1_startup() : activates the closure and gray-eas-rates
- subroutine m1_read_Weakhub() : reads in the weakhub table with gray weak rates and saves the tables with logarithms of fluid variables and opacities

- subroutine m1_get_wavespeeds(ixI^L, ixO^L, idim, wprim, x, lambda, qtC) : here the wavespeeds for the evolved m1-variables are computed which are needed in the finite_volume.
  The logic of calls is:
  finite_volume -> get_cmax -> get_lambda -> m1_get_wavespeeds
  and
  finite_volume -> get_cbounds -> get_lambda -> m1_get_wavespeeds
  The equations for the wavespeeds are:

$$\lambda^{(\pm)} := d_{\mathrm{th}}\lambda_{\mathrm{th}}^{(\pm)} + d_{\mathrm{TH}}\lambda_{\mathrm{TH}}^{(\pm)}, \tag{9}$$

  where

$$\lambda_{\mathrm{TH}}^{(\pm)} := \min\left(-\beta^j + p^j, r^j\right), \tag{10}$$

$$p^j := \frac{\alpha v^j}{W}, \tag{11}$$

$$r^j := -\beta^j + \frac{2W^2 p^j \pm \sqrt{\alpha^2\gamma^{jj}(2W^2+1) - 2(Wp^j)^2}}{2W^2+1}, \tag{12}$$

  and

$$\lambda_{\mathrm{th}}^{(\pm)} := -\beta^j \pm \alpha\frac{F^j}{\sqrt{F^i F_i}}. \tag{13}$$

- subroutine m1_get_fluxes(ixI^L, ixO^L, idim, x, wprim, flux, qtC) : here the fluxes for the evolved m1-variables are computed, which are needed in the finite_volume.
  The logic of calls is:
  finite_volume -> get_flux -> m1_get_fluxes

- subroutine m1_correct_asymptotic_fluxes(ix^D, ixI^L, tvdlfeps1, dxdim, idims, wLC, wRC, fC, fLC, fRC, cminC, cmaxC, wradimpl, qtC) : this routine takes the via HLL computed fluxes and adds corrections with the term $A = 1/\kappa/\Delta x$ for the diffusive limit. In this routine the user can make a choice from m1list for the correction of fluxes.
  The logic of calls is:
  finite_volume -> get_Riemann_flux_hll -> m1_correct_asymptotic_fluxes
  The expression for the fluxes is:

$$\tilde{F}^j := \sqrt{\gamma}\alpha \begin{pmatrix} \tilde{n}\left(W\left(v^j - \frac{\beta^j}{\alpha}\right) + \frac{H^j}{J}\right) \\ F^j - \frac{\beta^j E}{\alpha} \\ P_i^j - \frac{\beta^j F_i}{\alpha} \end{pmatrix} \tag{14}$$

- subroutine m1_add_geometrical_sources(ixI^L,ixO^L,x,wprim, wcons, qdt, d_k_gamma_ij, Kij_local,qtC) : here the geometrical source terms are added for the evolution of the m1-variables
  The logic of calls for nonDYSP is:
  finite_volume -> addgeometry -> m1_add_geometrical_sources
  The logic of calls for DYSP is:
  finite_volume -> addgeometry -> addgeometry_source_full_gr -> m1_add_geometrical_sources
  The expression for the geometrical sources is:

$$\tilde{S} := \sqrt{\gamma} \begin{pmatrix} 0 \\ \alpha P^{ij} K_{ij} - F^i \partial_i \alpha \\ -E \partial_i \alpha + F_j \partial_i \beta^j + \alpha P^{jk} \frac{\partial_i \gamma_{jk}}{2} \end{pmatrix} \tag{15}$$

- subroutine m1_add_collisional_sources(dtfactor,qdt, qtC, x, wcons, wold, wradimpl,ixI^L) : adds the collisionals sources to the implicit timestep of the time-evolution of m1-variables and also updates the fluid vars.
  The logic of calls is:
  advect -> global_implicit_update -> implicit_update -> m1_add_collisional_sources

The colliisonal sources are:

$$\tilde{G} := \sqrt{\gamma}\alpha \begin{pmatrix} N \\ \tilde{G}^\alpha \end{pmatrix} := \sqrt{\gamma}\alpha \begin{pmatrix} N \\ -S^\mu n_\mu \\ S^\mu \gamma_{\mu i} \end{pmatrix} = \sqrt{\gamma}\alpha \begin{pmatrix} Q^{\mathrm{nr}} - \kappa_a^{\mathrm{nr}} \tilde{n} \\ W\left[Q^{\mathrm{er}} + \kappa_s^{\mathrm{er}} J - \kappa^{\mathrm{er}}(E - F^i v_i)\right] \\ W\left(Q^{\mathrm{er}} - \kappa_a^{\mathrm{er}}\right) v_i - \kappa^{\mathrm{er}} \gamma_{\mu i} H^\mu \end{pmatrix} \tag{16}$$

4.3 Closure

The modules discussed here:
mod_m1_closure
mod_m1_closure_functions
mod_rootfinding

The closure is necessary to update the m1-variables in the Eulerian (N, E, F_i) and fluid frames (n, J, H_i) and to compute the pressure tensor P_{ij} in the optically thin and thick limit.
The module **mod_m1_closure**comes with a few routines:

- procedure(sub_m1_closure_func), pointer :: m1_closure_func => null()
  and also
  procedure(sub_m1_closure_deriv), pointer :: m1_closure_deriv => null()
  : these are pointers to the closure-function which will be assigned when calling subroutine m1_closure_type_activate() which is called inside of

subroutine m1_closure_activate()
The logic of calls is:
m1_startup -> m1_closure_activate -> m1_closure_type_activate -> m1_closure_func -> sub_m1_closure_func

- subroutine m1_update_closure(metricM1,wprim,x,ixI^L,ixO^L, species, get_zeta, Gamma,Hup, Jrad, Press, chi, W, vel, zetaOut,FilePrint) : this routine updates the closure on the whole block ixO^S and can even give out the optional arguments Gamma, Hup, Jrad, Press, chi, W, vel besides the standard m1-variables inside wprim()-array in the Eulerian frame.
  Note: this routine uses the wprim(m1-vars) devided by the metric determinant sqrtg and fills up the stateM1-class after doing the update pointwise in another subroutine.
  The logic of calls is:
  m1_get_wavespeeds -> m1_update_closure and
  m1_get_fluxes -> m1_update_closure and
  m1_add_geometrical_sources -> m1_update_closure and
  m1_add_collisional_sources -> m1_update_closure

- subroutine m1_update_closure_ixD(stateM1, metricM1, ix^D, get_zeta, get_vel_impl) : this routine updates the closure pointwise
  Note: the velocity coming into the routine is $v^i * W$ and the velocity coming out of the routine with stateM1%vel is the 3-velocity v^i
  The logic of calls is:
  Note: this routine uses a brent-rootfinder to find the Eddington-factor ξ
  m1_update_closure -> m1_update_closure_ixD

- function xi(z) : this is the function of which we need to find the root to get the Eddington-factor xi. It is contained in the m1_update_closure_ixD routine and thus can access stateM1

- function dxidz(z) : this is the derivative of the function of which we need to find the root to get the Eddington-factor xi. It is contained in the m1_update_closure_ixD routine and thus can access stateM1

The module **mod_m1_closure_functions** contains the m1_closure_func() and m1_closure_deriv() here linked to the Minerbo-closure or Levermore-closure and its derivative:

- function minerbo_closure(z) : Minerbo closure as function of Eddington factor
  The logic of calls is: m1_update_closure -> m1_update_closure_ixD -> rootfinding_brent -> xi -> minerbo_closure

- function minerbo_deriv(z) : derivative of the Minerbo closure as function of Eddington factor
  The logic of calls is:
  m1_update_closure -> m1_update_closure_ixD -> rootfinding_brent -> dxidz -> minerbo_deriv

- function Levermore_closure(z) : Levermore closure as function of Eddington factor
  The logic of calls is: m1_update_closure -> m1_update_closure_ixD -> rootfinding_brent -> xi -> Levermore_closure

- function Levermore_deriv(z) : derivative of the Levermore closure as function of Eddington factor
  The logic of calls is:
  m1_update_closure -> m1_update_closure_ixD -> rootfinding_brent -> dxidz -> levermore_deriv

4.4 Collisional

The modules discussed here:
advance
mod_m1
mod_m1_collisional
mod_m1_internal
mod_rootfinding

The collisional sources in the module **mod_m1_collisional**, which is important for the imlicit timestep of the solver. Here are the routines inside the module:

- subroutine m1_get_implicit_collisional_sources(wrad, metricM1, vel_, eas_,ixI^L,ix^D, N, sources, qdt, speciesKSP, dtfactor) : The closure is updated and a rootfinding for the m1-variables in a function with saved explicit part state and collisional part is started. The rootfinding is performed via a multidimensional rootfinding Newton Raphson scheme. Then the sources are computed again and also the source for the number density.
  The logic of the calls is:
  m1_add_collisional_sources -> m1_get_implicit_collisional_sources
  This subroutine contains a few other functions: [! beginning of contains

- function m1_sources_func(z_vec) : This function is the one we are trying to find to root of to get U^*. It includes the explicit part $\tilde{X}\left(U^{(n)}\right)$ and the collisional part $\tilde{G}\left(U^*\right)$:

$$0 = \tilde{X}\left(U^{(n)}\right) + \tilde{G}\left(U^*\right) - \tilde{U}^* \tag{17}$$

  The logic of the calls is:
  m1_add_collisional_sources -> m1_get_implicit_collisional_sources -> rootfinding_global_multid_newton_raphson -> m1_sources_func

- function m1_sources_jacobian(z_vec) : Contains the expressions of the jacobian of the sources-function needed for the rootfinder

The logic of the calls is:
m1_add_collisional_sources -> m1_get_implicit_collisional_sources -> rootfinding_global_multid_newton_raphson -> m1_sources_jacobian

- function m1_sources_min(z_vec) : Contains some minimal value of the sources-function needed for the rootfinder
  The logic of the calls is:
  m1_add_collisional_sources -> m1_get_implicit_collisional_sources -> rootfinding_global_multid_newton_raphson -> m1_sources_min
- function m1_collisional_sources(z_vec) : Contains the expressions of the actual collisional sources for the different m1-variables. Is needed for the rootfinder, which calls m1_sources_func
  The logic of the calls is:
  m1_add_collisional_sources -> m1_get_implicit_collisional_sources -> rootfinding_global_multid_newton_raphson -> m1_sources_func -> m1_collisional_sources
- function m1_collisional_jacobian(z_vec) : Contains the expressions of the jacobian of the collisional sources needed for the rootfinder, which calls m1_sources_jacobian
  The logic of the calls is:
  m1_add_collisional_sources -> m1_get_implicit_collisional_sources -> rootfinding_global_multid_newton_raphson -> m1_sources_jacobian -> m1_collisional_jacobian
- Linearized versions of all 4 upper routines
  [! end of contains

- subroutine get_iter_initguess(z_vec) : Gives an initial guess for the rootfinder assuming first order of the sources and optically thick regime to update the closure and get the m1-variables in Eulerian frame.
  The logic of the calls is:
  m1_add_collisional_sources -> m1_get_implicit_collisional_sources -> get_iter_initguess
- subroutine m1_get_implicit_n_source(N, dYE) : This routine performs the implicit update of the number density including collisional sources for the neutrino continuity equation
  The logic of the calls is:
  m1_add_collisional_sources -> m1_get_implicit_collisional_sources -> rootfinding_global_multid_newton_raphson -> m1_sources_func -> m1_collisional_sources

4.5 Rates

The modules discussed here:
advance

amrvacphys
mod_m1_eas
mod_m1_eas_param
mod_m1_eas_microphysical_grey
mod_m1_eas_tests
mod_m1_fermi
mod_m1_constants
mod_m1_internal

4.5.1 Weakhub

The modules discussed here:
mod_Weakhub_reader
mod_weakhub_parameters

4.5.2 Optical depth

The modules discussed here:
mod_m1_tau_optical
mod_m1_eas_microphysical_gray

4.5.3 Beta equilibrium

The modules discussed here:
mod_m1_beta_eq
mod_m1_eas
mod_rootfinding

4.6 Outflow

The modules discussed here:
mod_healpix_det
mod_post_process
mod_m1_outflow

5 Parameters: m1lists

```
&m1list
    m1_closure_type = "Minerbo" !"Levermore"
    fileWeakhub = '/mnt/rafast/mcassing/Weakhub/dURCA_ &
```

```
        convenforbetaproc_DD2_grey_npe_20240723_Weakhub_grey.h5'
    m1_actual_speeds = T
    m1_radice_speeds = F
    m1_parabolic = F
    m1_frad_A2_A3 = F
    m1_frad_A2_A = T
    m1_erad_LLF = F
    m1_frad_LLF = F
    m1_fluid_backreact = F
    m1_use_muons = F
    TESTqtC = 1.0d+5
indent m1_2_eas_updates = T
&end
```

```
&eoslist
    use_realistic_mp_table = T
&end
```

6 Definitions

```
#define M1
#undefine M1_EXPLICIT
#define M1_RATES
#undefine M1_SPEED1
#undefine M1_TAU_OPT

#undefine M1_FLAT_METRIC
```

7 Tests